
Lecture 14: Support Vector Machine Part 2

Md. Shahriar Hussain
ECE Department, NSU



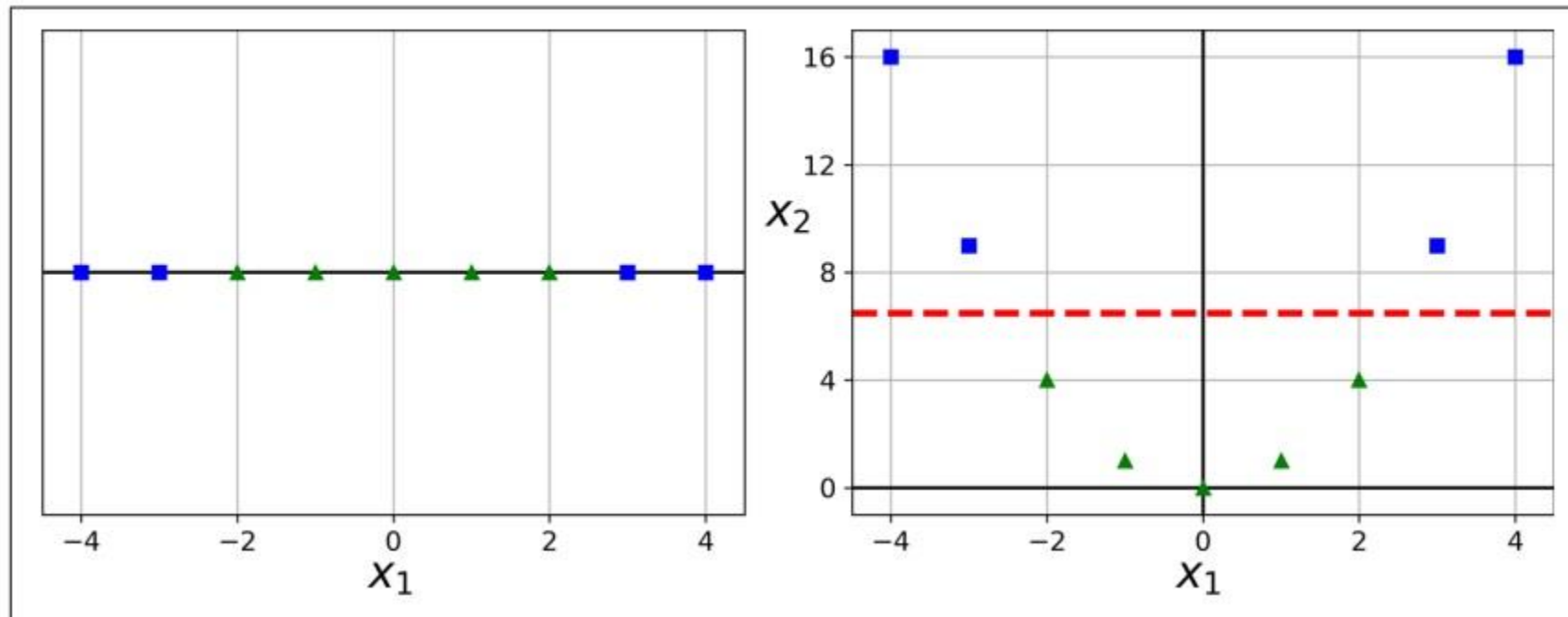
Nonlinear SVM Classification

- Although linear SVM classifiers are efficient and work surprisingly well in many cases, many datasets are not even close to being linearly separable
- One approach to handling nonlinear datasets is to add more features, such as polynomial features



Nonlinear SVM Classification

- it represents a simple dataset with just one feature, x_1 . This dataset is not linearly separable.
- But if we add a second feature $x_2 = (x_1)^2$, the resulting 2D dataset is perfectly linearly separable

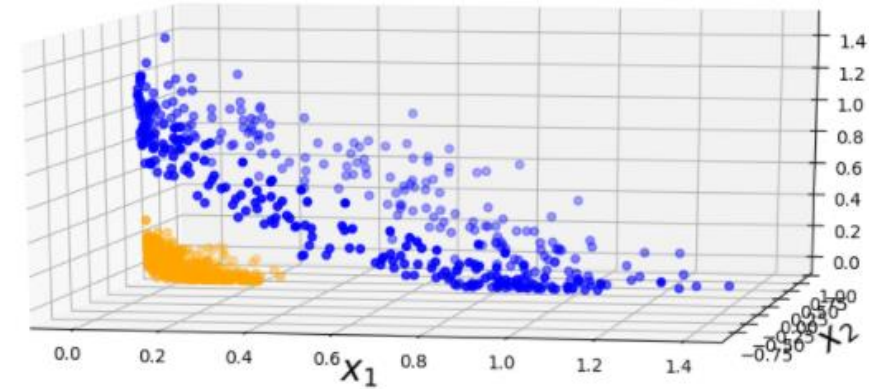
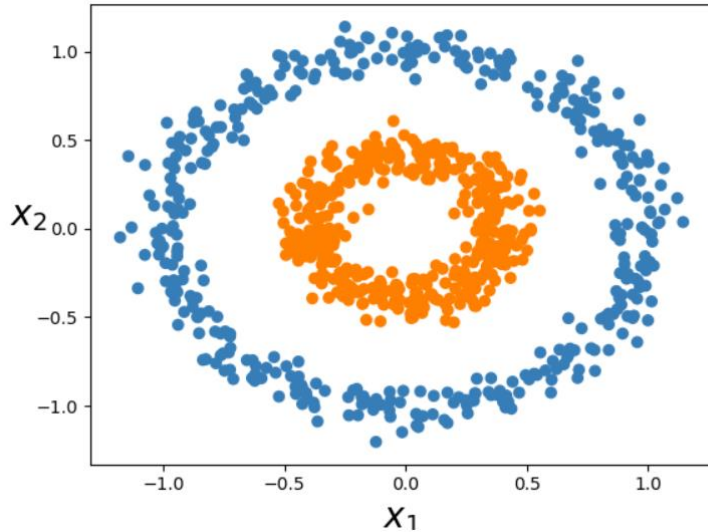


SVM Kernel: Nonlinear Transformations – 1D → 2D

- If data isn't linearly separable in the original space
 - map it to a higher dimensional feature space
- Kernel trick: convert low dimension to higher dimension
- If original data isn't linearly separable
 - Apply nonlinear transformation as $\phi(x) = x^2 \rightarrow$ we can draw a linear decision boundary!
 - Several functions possible to map the data to higher dimensions



Nonlinear Transformations – 2D → 3D



Linearly separable data in 3-d after applying the 2nd-degree polynomial transformation

- Applying a second-degree polynomial mapping →

$$\phi(\mathbf{x}) = \phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} x_1^2 \\ x_1 x_2 \\ x_2^2 \end{pmatrix}$$

- Note: Transformed vector is 3D!

Kernel trick for a second-degree polynomial mapping

- Applying a second-degree polynomial mapping $\rightarrow$

$$\phi(\mathbf{x}) = \phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} x_1^2 \\ x_1 x_2 \\ x_2^2 \end{pmatrix}$$

If $\mathbf{a}$ and $\mathbf{b}$ are some 2D vectors,

$$\begin{aligned} \phi(\mathbf{a})^\top \phi(\mathbf{b}) &= \begin{pmatrix} a_1^2 \\ \sqrt{2} a_1 a_2 \\ a_2^2 \end{pmatrix}^\top \begin{pmatrix} b_1^2 \\ \sqrt{2} b_1 b_2 \\ b_2^2 \end{pmatrix} = a_1^2 b_1^2 + 2a_1 b_1 a_2 b_2 + a_2^2 b_2^2 \\ &= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^\top \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = (\mathbf{a}^\top \mathbf{b})^2 \end{aligned}$$

Kernel trick for a second-degree polynomial mapping

- The dot product of the transformed vectors is equal to the square of the dot product of the original vectors: $\phi(\mathbf{a})^\top \phi(\mathbf{b}) = (\mathbf{a}^\top \mathbf{b})^2$
- if you apply the transformation ϕ to all training instances $\rightarrow$ the dot product of $\phi(\mathbf{x}^{(i)})^\top \phi(\mathbf{x}^{(j)})$
- But if ϕ is the second-degree polynomial transformation, we can replace this dot product of transformed vectors simply by $((\mathbf{x}^{(i)})^\top (\mathbf{x}^{(j)}))^2$
- So, we don't need to transform the training instances at all; just need to replace by the dot product of the vectors and make square
- this trick makes the whole process much more computationally efficient



Kernels

- In Machine Learning, a *kernel* is a function capable of computing the dot product $\phi(\mathbf{a})^T \phi(\mathbf{b})$
- It takes vectors in the original lower dimensional space as inputs, and returns the dot product of the transformed vectors in the higher dimensional feature space
- Dot product of transformed vectors is equal to the square of the dot product of the original vectors
- The kernel is any function that's capable of computing the dot product without having to compute (or know at all) the transformation ϕ



Common kernels

- The function $K(\mathbf{a}, \mathbf{b}) = (\mathbf{a}^\top \mathbf{b})^2$ is a second-degree polynomial kernel
- Some common kernels-

Linear: $K(\mathbf{a}, \mathbf{b}) = \mathbf{a}^\top \mathbf{b}$

Polynomial: $K(\mathbf{a}, \mathbf{b}) = (\gamma \mathbf{a}^\top \mathbf{b} + r)^d$

Gaussian RBF: $K(\mathbf{a}, \mathbf{b}) = \exp(-\gamma \|\mathbf{a} - \mathbf{b}\|^2)$

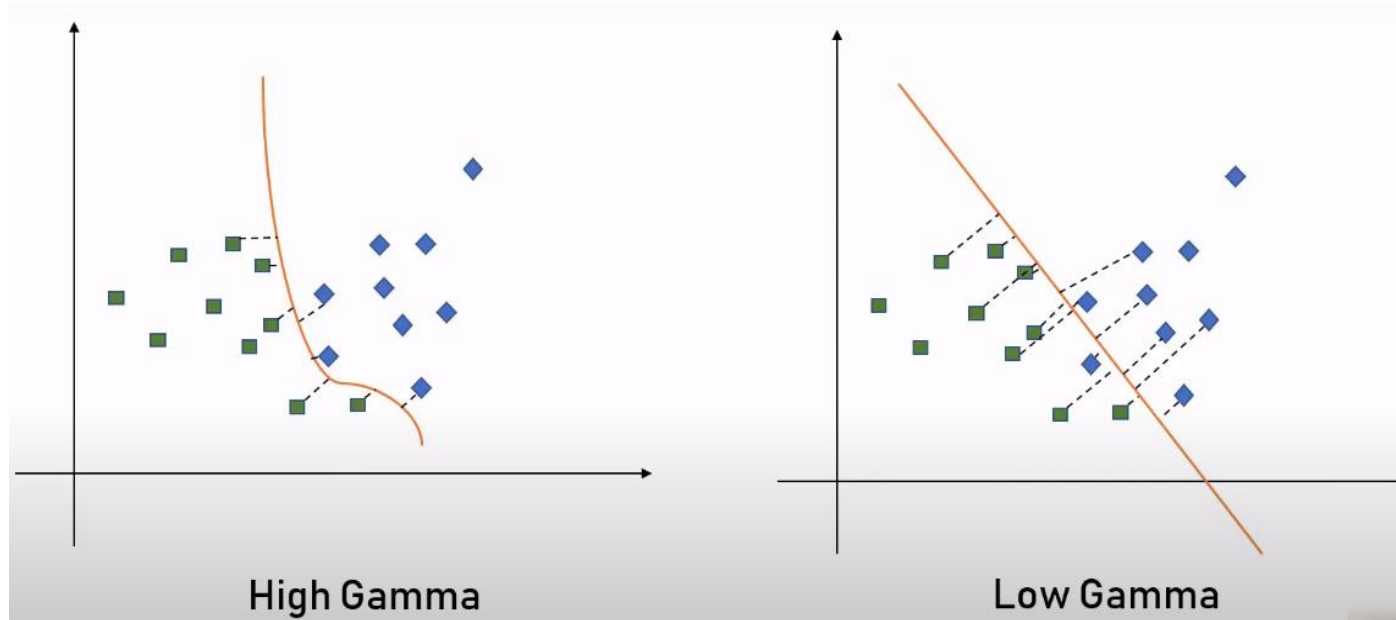
Sigmoid: $K(\mathbf{a}, \mathbf{b}) = \tanh(\gamma \mathbf{a}^\top \mathbf{b} + r)$

SVM Hyperparameters

- **C:** float, default=1.0: Regularization/Hinge Loss parameter
- **Kernel:** 'linear', 'poly', 'rbf', 'sigmoid', default='rbf'
- Specifies the kernel type to be used in the algorithm. If none is given, 'rbf' will be used
- **Degree (d):** default=3: Degree of the polynomial kernel function ('poly')
- **Gamma:** Controls how many data points will be considered for decision boundary
- **$r = \text{coef0}$** = Independent bias term in kernel function. It is only significant in 'poly' and 'sigmoid'



SVM Gamma



- **High gamma:** Only the **closest** points (**less points**) will be used to draw the decision boundary
- **Low gamma:** **Far away** points (**more points**) will be used to draw the decision boundary

Support Vector Regression

- the SVM algorithm is versatile: not only does it support linear and nonlinear classification, but it also supports linear and nonlinear regression
- To use SVMs for regression instead of classification, the trick is to reverse the objective:
 - instead of trying to fit the largest possible street between two classes while limiting margin violations, **SVM Regression** tries to **fit as many instances as possible** on the street while limiting margin violations



Support Vector Regression

- Margin/Width of the street is controlled by a hyperparameter (max error = ϵ)
- Tune epsilon to gain the desired accuracy of our model
- Hard SVR Constraint:
 $|y_i - \tilde{y}_i| \leq \epsilon$

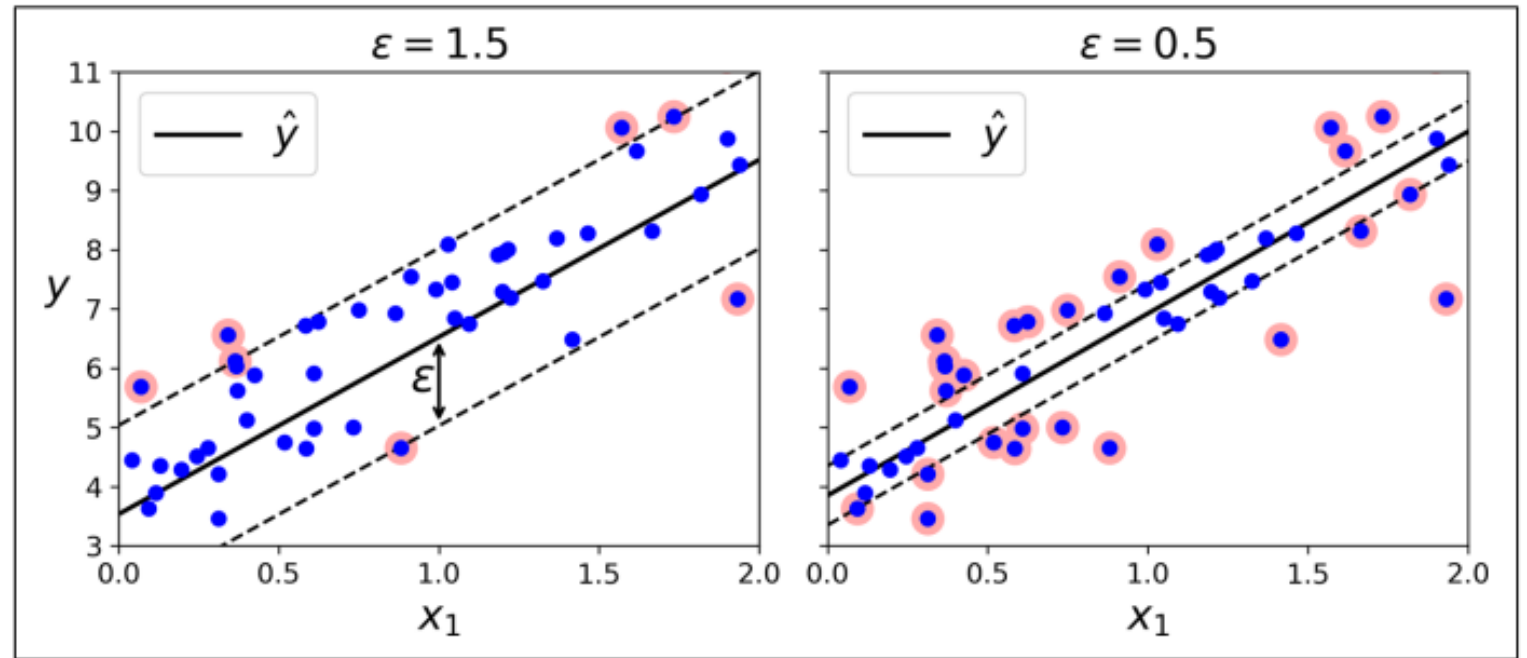
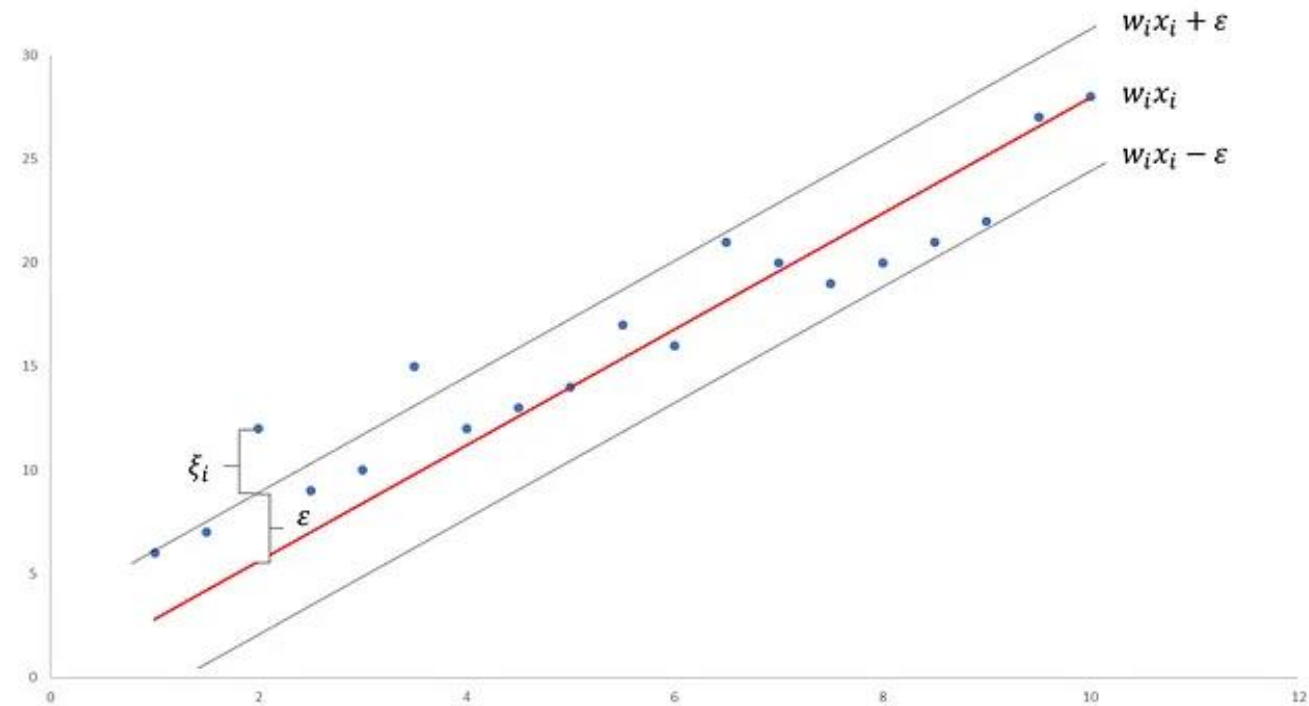


Figure 5-10. SVM Regression

Support Vector Regression

- Soft SVR Constraint:
- $|y_i - \tilde{y}_i| < \epsilon + C \sum_{i=1}^m \zeta^{(i)}$



SVM For Outlier/ Anomaly Detection

- One-Class SVM: unsupervised model for anomaly or outlier detection.
- Unlike the regular supervised SVM, the **one-class SVM does not have target labels** for the model training process.
- Learns the boundary for the normal data points and identifies the data outside the border to be anomalies
- Hyperparameter ν is to specify the percentage of anomalies. $\nu=0.01$ means that we have around 1% outliers in the dataset



SVM For Outlier/ Anomaly Detection

- # of outlier = 6

